# Table of Contents

Introduction....3
Step 1 - Object Masking....4
Step 2 YOLOv11 Training....9
Step 3 Model Evaluation....13
Conclusion....15

# List of Figures

Figure 1: Original Image....4
Figure 2: Rotated and Gray-scaled image....5
Figure 3: Threshold Image....6
Figure 4: Canny Edge Detection Image....7
Figure 5: Detected Contours....8
Figure 6: Masked Image....8
Figure 7: Extracted Image....9
Figure 8: Normalized Confusion Matrix....10
Figure 9: Precision vs Confidence Curve....11
Figure 10: Precision vs Recall Curve....12
Figure 11: Arduino Mega Predications....13
Figure 12: Arduino Uno Predications....14
Figure 13: Raspberry Pi Predications....15

## Introduction

PCB inspection is a very important step in electronics manufacturing, where accurate identification of components is necessary for assembly verification, fault detection and quality control. This project delves into PCB-component detection using classical image processing techniques and deep learning. In step 1, thresholding, edge detection and contour extraction were used to isolate the PCB from its background. In step 2, a YOLOv11 model was trained, which enables the network to localize and classify components across many PCB images. Finally, in step 3, the model was evaluated on unseen images, which allowed one to examine its performance through a normalized confusion matrix, a precision-confidence curve, and a precision-recall curve

# Step 1 - Object Masking

In this step, classical computer vision techniques were used. The goal of this was to use thresholding, edge and contour detection, as well as masking, to isolate the motherboard image from its background. The original image can be seen below

**Figure 1:** Original Image

## Image Standardization and Grayscale Conversion

The motherboard image was first imported into the code by using its directory. After loading the image, it was rotated 90 degrees clockwise to align it with the example image in the project outline. This rotated image was then converted to gray scale using cv2 because binary thresholding, edge detection and contour extraction operate on single-channel intensity images. In Figure 2 below, the rotated and grayscaled image can be seen.

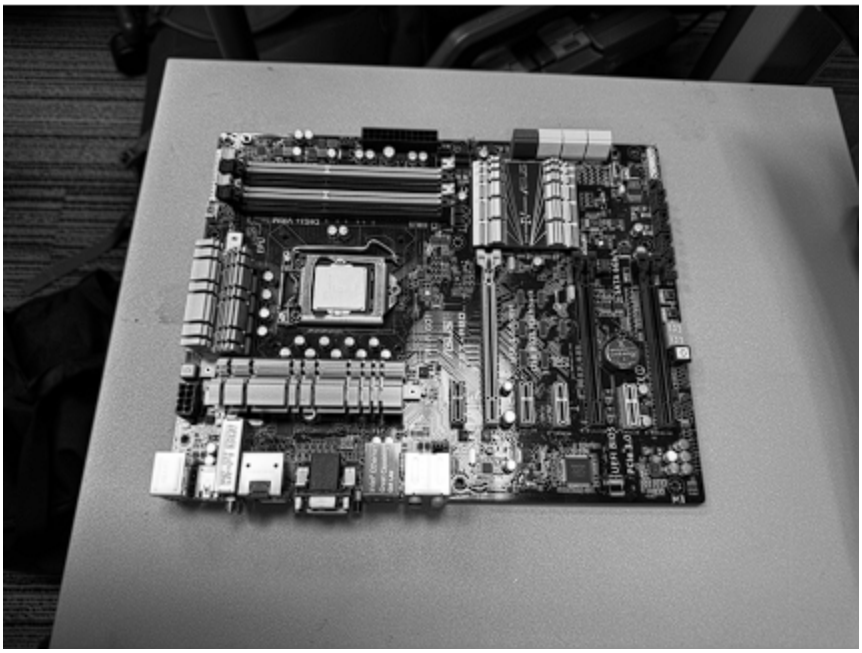

**Figure 2:** Rotated and Gray-scaled image

## Thresholding

A binary Threshold was then applied to the grayscale image to separate the image into two intensity regions, these being the foreground and background. Pixels brighter than the chosen threshold become white, and the darker pixels become black. This converts the image into a binary mask, which makes shape extraction easier.

The threshold intensity value chosen was 100. This was selected through visual experimentation. Setting the threshold lower resulted in background noise, while setting it higher caused darker PCB regions to disappear. The value of 100 provided a balance allowing bright PCB regions to become white while darker background regions became black.

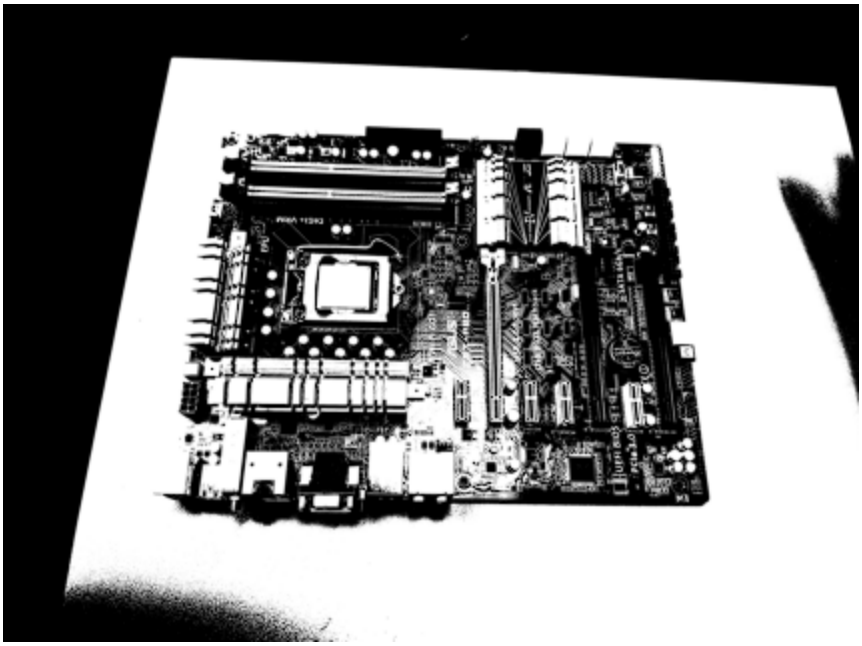

**Figure 3:** Threshold Image

## Gaussian Blur

A Gaussian blur applies a smoothing filter which reduces image noise and small pixel variations. This helps to prevent false edges and small noisy contours from appearing, which will help in later steps.

The kernel size of the Gaussian blur was chosen to be 5x5 as it provided an optimal trade-off between noise reduction and edge preservation. A smaller 3x3 kernel size did not remove enough random noise, while a larger 7x7 kernel blurred PCB edges aggressively.

## Edge Detection: Corner Detection

Canny edge detection was used to detect strong intensity changes that correspond to object boundaries. It highlights important structural features by outlining component boundaries and board edges. This results in a clean map of significant edges that will be used for further processing.

Canny edge detection takes 2 parameters, these being a low threshold and a high threshold. To satisfy these parameters, the low threshold was chosen to be 100, while the high threshold was chosen to be 200.

- A low threshold of 100 allows moderate to strong edges to be included, such as connector borders and Outer PCB boundaries.
- A high threshold of 200 makes sure that only truly strong edges trigger edge propagation, which prevents noise edges from forming closed contours.

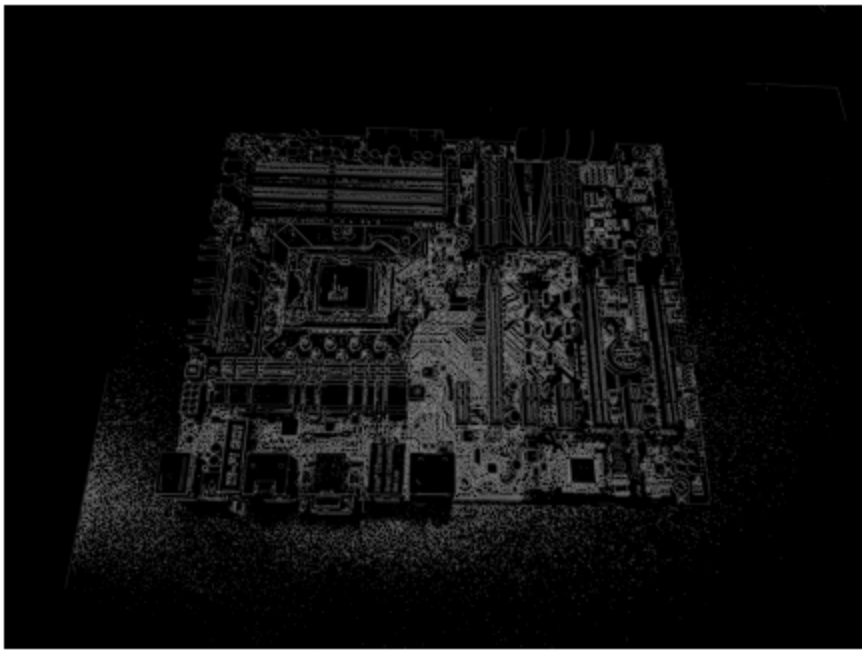

**Figure 4:** Canny Edge Detection Image

## Edge Detection: Contour Detection

*Note: Although edge detection can be done by either the Canny edge or the contours. Both options were experimented with here, which resulted in contour detection having better results.

Contours are continuous boundaries which are around connected white regions in a binary image. Detecting the contours in the image can help to identify the outer shape of the motherboard and distinguish it from internal components. Furthermore, not all detected contours are necessary and need to be filtered; these would be contours that are too small, have unrealistic aspect ratios or have noise. This ensures that only the best contour is used.

After finding all the contours, a loop was used to examine each contour using a criterion to decide whether it could represent the motherboard. The loop calculates the area and size for each contour and keeps track of the largest one. By the end of the loop, the largest contour is selected since the PCB is the biggest meaningful object in the image. This contour was then drawn on the motherboard image.

<u>Constants in loop</u>

Minimum width/height = 150 pixels

- This was chosen as values smaller than this are just screw holes or noise. This prevents misclassification of small contours as the board boundary.

Maximum area threshold = 16,000,000 pixels

- This value was found by calculating the approximate PCB area from the image resolution. It was found that this value effectively prevents large contours from being selected.

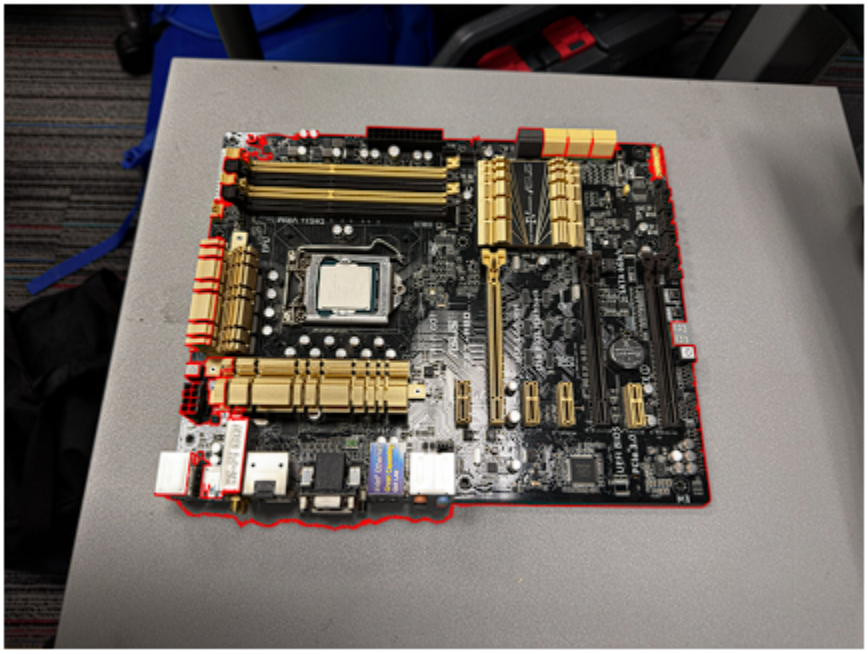

**Figure 5:** Detected Contours

## Masking

A binary mask is made from the selected PCB contour using bitwise operations. This helps to highlight only the PCB region while setting all external areas to black.

**Figure 6:** Masked Image

### Extracted Image

**Figure 7:** Extracted Image

The final extracted image is seen above. It shows the PCB on a black background, indicating that the object masking was successful in isolating the image and removing unnecessary background details. Furthermore, comparing Figure 7 to the example image in the project outline, it can be seen that they are almost identical, further proving that the object masking was done very well. However, it is not perfect; there is an area in the bottom left of the image that was not extracted. This may have been because the area's background resembles that of the motherboard, which makes it hard to choose a signal threshold value that perfectly separates it. This could be improved by using adaptive thresholding, which adjusts the threshold based on the local pixel intensity.

## Step 2 YOLOv11 Training

The YOLOv11 model was trained on the provided dataset, which contains 13 PCB component classes. The training was done for 150 epochs using an image size of 1200x1200 with a batch size of 5. The image size was kept as large as possible to improve the detection of very small components, and a smaller batch size was used due to the limited GPU memory in Google Colab. Throughout the training, the box loss, classification loss and DFL loss decreased, which shows that the model successfully learned to localize and classify the components. After training, the model was automatically saved by YOLO in runs/detect for later use in evaluation.

## Normalized Confusion Matrix

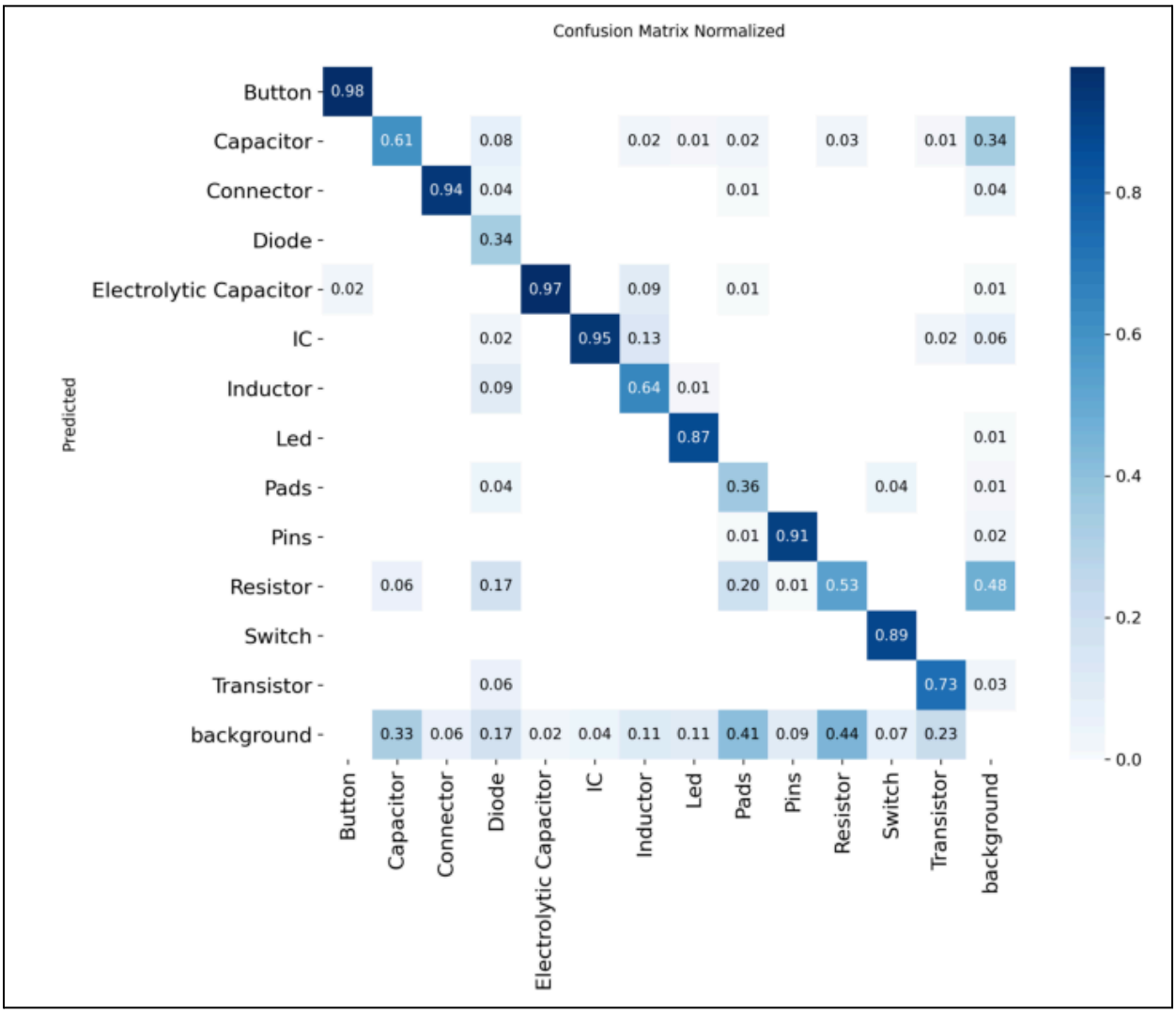


**Figure 8:** Normalized Confusion Matrix

The normalized confusion matrix above shows a visual summary of how accurately each class was predicted. Along the diagonal, higher values indicate high classification accuracy. From the matrix, it can be seen that the model showed strong performance for buttons, connectors, electrolytic capacitors, ICs, switches, LEDs, and pins. Some of these components were physically larger or more distinct, which explains why they were classified accurately. However, other classes show weaker accuracies along the diagonal; these include capacitors, resistors, diodes, inductors, and pads. These components are small, look similar and are more often clustered close together on the motherboard, which makes it more difficult for the model to differentiate. Furthermore, in the rightmost column, it can be seen that resistors and capacitors were sometimes mistaken for the background, with percentages of 48% and 34% respectively. Also, from the bottom row, it can be seen that the background was sometimes mistaken for components. This may be due to certain board textures and lighting. Overall, from the diagonal

dominance, it can be said that the model was fairly good at classifying components; however, there was still some confusion between components indicated by the dark squares on the off-diagonal.

## Precision-Confidence Curve

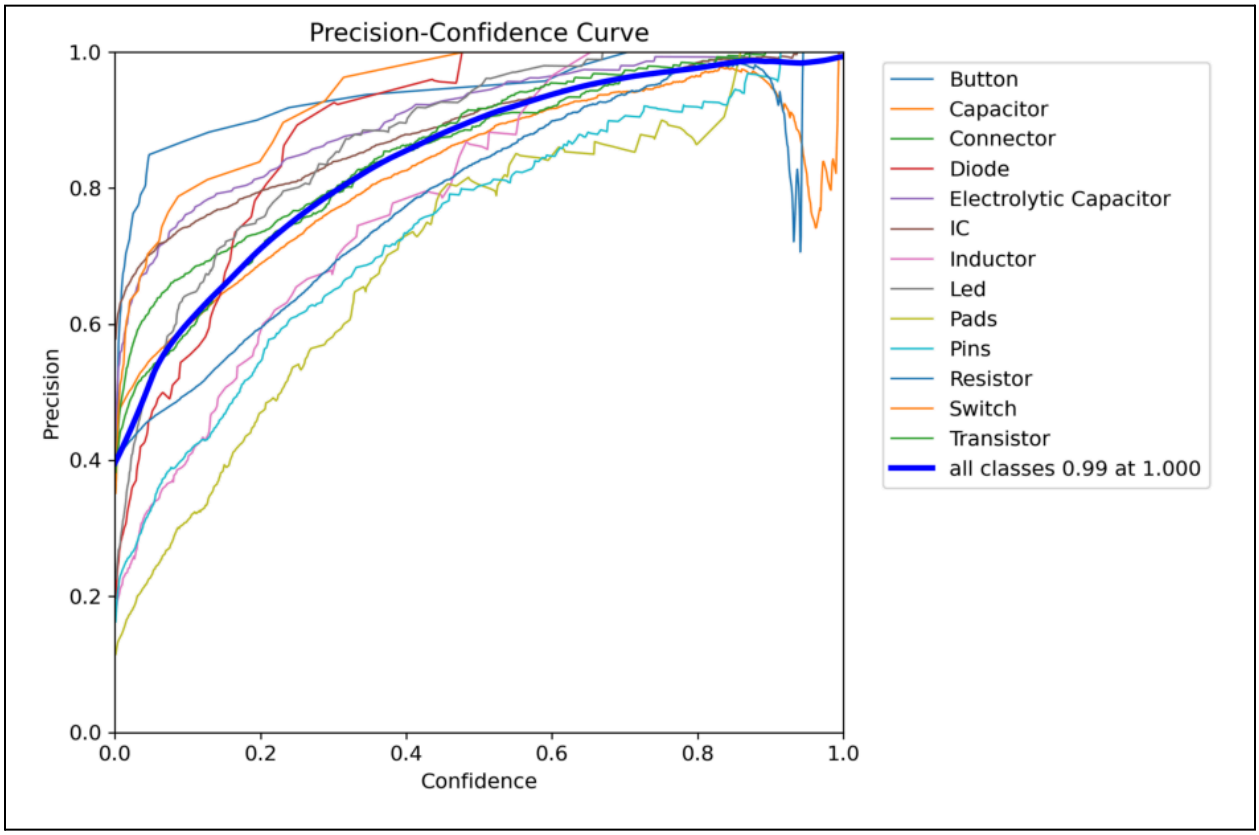


**Figure 9:** Precision vs Confidence Curve

The precision vs confidence curve delves into how the model's precision changes as the confidence threshold increases. At very low confidence values, the model tends to have many detections with some being false positives, which causes reduced precision. Moreover, as the confidence threshold increases, the model becomes more selective and the precision improves. It can be seen that most classes achieved high precision values at mid-high confidence levels, with the combined "all classes" curve reaching close to 0.99 at high confidence. This is a good trend and is very desirable in the model. However, there were still some classes that had more variability in their curves, these being pads, resistors, diodes, and inductors. This may indicate that the model was less certain when distinguishing these components, as they look similar.

## Precision-Recall Curve

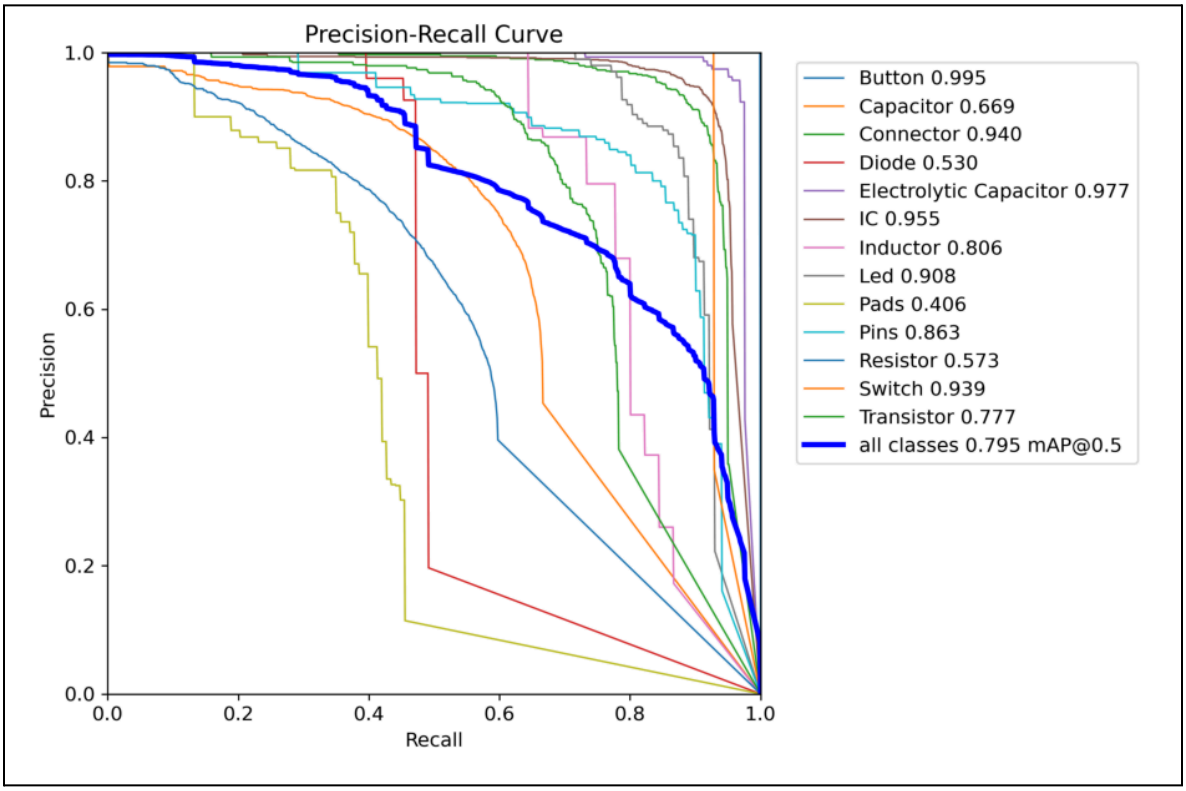


**Figure 10:** Precision vs Recall Curve

The precision vs recall curve shows a different perspective of the model's performance. Classes that have curves that remain near the top right corner, such as the button, connector, IC, switch, LED, pins and electrolytic capacitor display both high precision and high recall. This means that the model detected most instances of these components and classified them correctly (few false positives and negatives). Weak performing classes such as pads, resistors, capacitors, diodes, and inductors showed lower recall. This means that the model often failed to detect these components consistently, even when its predictions were accurate. However, this type of behaviour is expected for small components, as they are hard to classify. From the legend on the right, it is seen that mAP@0.5 was 0.795. This means that the average precision across all classes at a threshold of 0.5 is 79.5% which indicates decent performance across all classes.

## Step 3 Model Evaluation

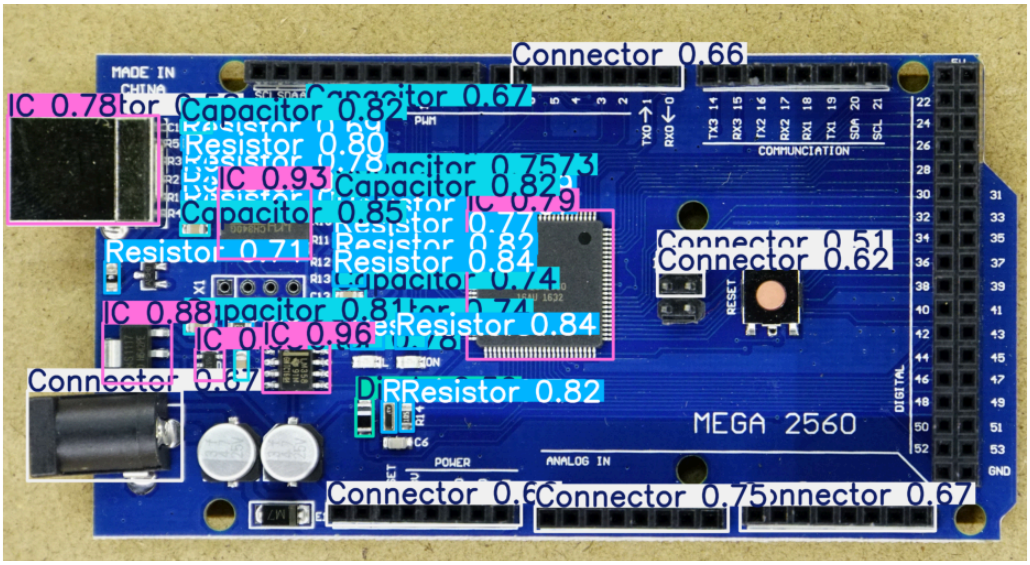


**Figure 11:** Arduino Mega Predications

The first test image is the Arduino Mega. Looking at Figure 11, the model did fairly well at detecting major components like connectors, ICs, resistors and capacitors. However, there were still some mistakes. Firstly, the connector on the right and the 2 on top were not detected at all. The pins on the left middle and the button on the right were also missed. A few resistors and capacitors were being confused with one another because of similar rectangular shapes and close spacing; however, there weren't too many. Lastly, the electrolytic capacitors in the bottom left were also missed, and one of the resistors was mistaken for a diode. Overall, despite these issues, the overall detection accuracy was relatively good.

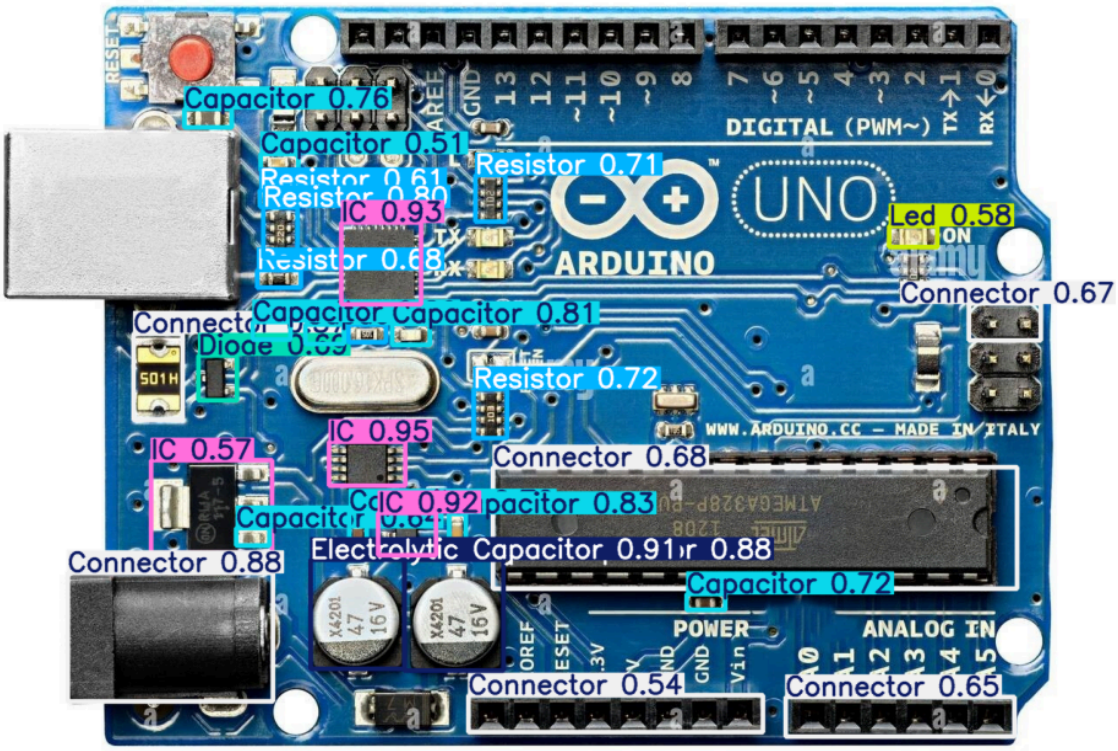


**Figure 12:** Arduino Uno Predications

The second testing image was the Arduino Uno. This image also showed similar detection tendencies as the previous image. Major components like the ICs, LED and electrolytic capacitor were detected with high confidence. However, some connectors are missing, including the one on the left, 2 at the top, and 3 more underneath the top ones. Similar to Figure 11, some resistors and capacitors were confused with one another. Furthermore, 2 capacitors were missed near the middle, and a transistor was missed on the right-hand side. Lastly, the model mistook an IC for a diode. The model did manage to detect the LED on the right, but it still missed the 2 in the middle. The electrolytic capacitor was detected correctly in this image, whereas it was missed in the first image. Overall, this image did decently well; however, it can be said that it performed worse than the first image in Figure 11.

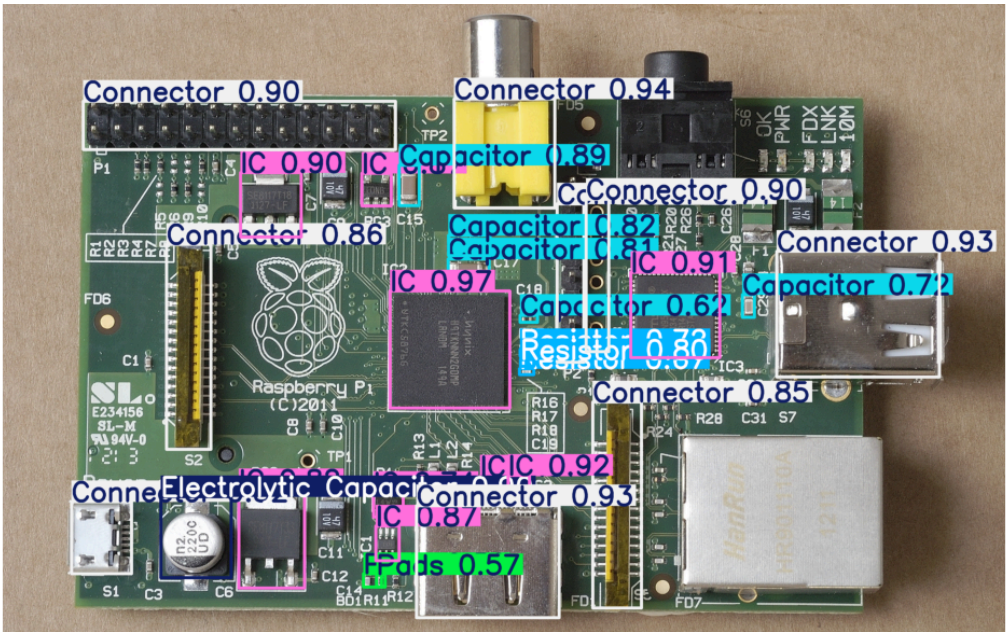


**Figure 13:** Raspberry Pi Predications

The last testing image is the Raspberry Pi. This image had a lot of very good predictions. The connectors, resistors, capacitors and ICs were all detected pretty well. There was less confusion between resistors and capacitors in this image compared to the other 2. However, there were still a few resistors missed in the top left and at the bottom middle; a resistor was mistaken for a pad. Furthermore, all of the ICs and connectors were correctly identified except for the audio jack connector. Overall, this image likely performed the best out of all 3, showing the cleanest detections and highest confidence scores with few misclassifications.

## Conclusion

Overall, the model performed well. From analyzing each image's predications, it can be inferred that image 3 performed the best, whereas image 2 was the worst. To increase the performance of the model, there are several methods that could be adopted. Firstly, one could increase the image size to greater than 1200 pixels, which would provide more pixel information for distinguishing smaller PCB components. This would help a lot with the resistor/capacitor confusion. However, this would require more VRAM and cause higher runtimes. Furthermore, one could increase the number of epochs to give the model additional time to refine its representations; however, this would also increase the runtime by a lot. Also, different components such as LEDS and pads were underrepresented in the dataset; including more images of these components would also result in better performance. Lastly, one could upgrade to a larger YOLO model, such as YOLOv11-S or YOLOv11-M.

Additionally, another way to detect some of the components that were completely missed is to lower the model's confidence threshold. In this project, the confidence level was set to 0.5, which is a commonly used balance point between precision and recall. This value ensures that the model only displays detections it is reasonably confident in, keeping the predictions clean and reducing false positives. Lowering the confidence threshold below 0.5 can increase recall and reveal smaller or less distinct components; however, this was not done for the final evaluation because it greatly increases false detections and makes the output cluttered.